PungentFunk Utilities

Architecture and Design Principles Bible

Package Modularity, Authoring Foundation, and Overlay Tray Doctrine Edition

Updated: 2026-05-10

Purpose

Stable architecture, package boundaries, Unity Editor implementation doctrine, verifiable UI preservation, shared package
service reuse, shared project scan pipeline doctrine, optional extension packaging, overlay tray and popup-surface doctrine, and the shared Authoring Data Foundation
required for concurrent document, board, and data-sheet development.

Update rule

Update this bible when product philosophy, package boundaries, UI doctrine, interoperability rules, source-truth rules, shared
service ownership, scan pipeline integration, authoring data contracts, extension package boundaries, overlay tray/windowing doctrine, AI-agent interpretation
rules, or documentation governance changes. Update snapshot audits separately whenever scripts change.

What changed in this edition

This edition keeps the Shared Systems and Project Scan Pipeline doctrine spine and applies the package-split and Authoring Foundation updates. It makes optional extension packages, bridge modules, package capability declarations, and Browser-versus-Editor separation first-class doctrine.

- Adds explicit Free/Core, themed extension, optional bridge, full bundle, and project adapter distribution rules.
- Adds the Core Anti-Sink Rule to prevent Core from absorbing lab-specific workflows.
- Adds Package Capability Declaration rules for registry descriptors and provider registration.
- Adds the Shared Authoring Data Foundation as a Core-owned model/contracts layer.
- Separates Authoring Browser, Rich Document Editor, Board/Whiteboard Editor, and Data Sheet Editor surfaces.
- Defines the Notes and Roadmap migration strategy: preserve the browser and integrations, deprecate only the old freeform
  writing panel.
- Adds safe missing-package behavior for optional editors and bridge integrations.
- Adds the package split map for Core, Full Bundle, Documentation and Authoring, Board/Graph, Data Sheet, Audit/Validation,
  Scene Authoring, Asset Production, Appearance/Colour/Texture, Audio, Debug, UI/Input Feedback, and Content Generation
  packages.
- Adds package split review rules for future Codex briefs and implementation passes.
- Adds Overlay Tray and Popup Window Doctrine: transient popup-style workflows should prefer shared overlay trays over detached popup windows when possible.

Central doctrine update

PungentFunk Utilities should not become a monolithic toolbox. Core owns stable contracts and package-wide infrastructure.
Extensions own concrete workflows. Bridges connect packages without hard dependencies. The Authoring Data Foundation must let documents, boards, sheets, notes, tokens, help topics, audit issues, utilities, and external targets share IDs, links,
metadata, previews, actions, and validation results without forcing every editor package to be installed. Transient popup-style workflows should prefer shared overlay trays over ad hoc detached popup windows when the task is contextual, lightweight, dismissible, and does not require a persistent dockable workspace.

Contents

0. How to use this bible

What this document is
This is the doctrine source for PungentFunk Utilities. It records architecture and package boundaries, Unity Editor

implementation doctrine, UI, accessibility, layout, messaging principles, dependency and optional integration rules, release-
hardening expectations, AI-agent interpretation rules, verifiable UI preservation rules, shared package service reuse rules,
shared project scan pipeline doctrine, overlay tray/windowing doctrine, and the shared authoring data contracts used by notes, documents, boards, sheets,
tokens, help, audit issues, utilities, and external targets.

It does not record volatile script counts, sprint tasks, one-off bug lists, per-PR implementation details, tutorial-level API
instructions, or exhaustive code examples. Implementation inventories belong in snapshot audits. Detailed how-to patterns
belong in pattern pages and lab-specific specs.

Reading paths

Reader	Read first	Then read
AI coding agent preparing an audit	Sections 0, 2, 3, 4, 5, 6, 10, 11, 12, 15, 21, 22, 27	Current source files, audit, relevant package map
AI coding agent preparing an implementation brief	Sections 0, 2, 3, 4, 8, 10, 11, 12, 14, 15, 22, 23, 27	Relevant lab spec, current source tree, path-specific instructions
AI coding agent adding scan, validation, page or authoring coverage, indexing, token, documentation, or authoring features	Sections 2, 3, 4, 11, 12, 14, 15, 18, 21, 22, 27	Shared scan pattern, page and Authoring Data Foundation pattern
Human implementer	Sections 1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 16, 18	Relevant lab spec and implementation pattern
Reviewer	Sections 5, 6, 8, 11, 12, 14, 15, 23, 24, 25, 26, 27	Changed files, active UI, validation steps
Designer or UX contributor	Sections 6, 7, 8, 9, 10, 13, 14	Visual examples, layout patterns, help/documentation patterns
Producer or planner	Sections 0, 1, 3, 4, 18, 20, 24, 27, 28	Roadmap and package release plan

Source-of-truth hierarchy

1. **The user current explicit instruction.**
2. **Current uploaded source files or current repository source tree.**
3. **The active UI as rendered in Unity, including visibility conditions, layout, scroll behavior, resizing behavior,** and interactive reachability.
4. **This design bible for durable doctrine.**
5. **The current snapshot audit for implementation state.**
6. **The current feature inventory for backlog/status.**
7. **Lab specs and pattern pages for detailed implementation guidance.**
8. **Historical briefs, older audits, old debriefs, and conversation notes.**

Implementation-state vocabulary

Term	Meaning
Implemented in source	Relevant classes, fields, methods, registrations, and call paths exist in current files.
Reachable in UI	A user can access the feature through the intended menu, window, panel, inspector, context menu, overlay, or shortcut without hidden prerequisites.
Visible in UI	The feature is displayed at usable sizes and is not obscured, clipped, buried, or only visible after accidental layout expansion.
Usable in UI	Controls are legible, interactable, not cramped beyond reasonable use, and provide expected feedback.
Verified	Source evidence, active UI evidence, and validation steps support the claim. Do not call a UI feature complete without all of these.

1. Scope and North Star

PungentFunk Utilities is a family of modular Unity editor/runtime utility assets, not a single monolithic toolbox. Each lab or
suite should be independently releasable, while the larger bundle exposes a cohesive control panel, shared visual
language, consistent interaction vocabulary, and optional cross-lab enhancements.

The core promise is simple: automate tedious game-development workflows without adding new tedium. Every tool should
be understandable at a glance, safe to try, powerful enough to scale through presets and profiles, and integrated into

the
Unity Editor surface where the user naturally needs it. A tool that technically exists but is hidden, inaccessible, cramped,
duplicated, compile-fragile, or visually degraded has not met the product promise.

Release model

Tier	Role	Rule
Free/Core package	Registry, launcher, theme, preferences, shared layout helpers, minimizer/tray, documentation/help opening contracts, shared scan/result contracts, provider registries, Developer Mode visibility contracts, token parser contracts, and Authoring Data Foundation contracts.	Keep Core small, stable, conservative, and boring. Core may provide a lightweight Authoring Browser shell and read/open functionality, but must not ship every concrete editor.
Standalone themed extension packages	Independently useful products such as Documentation and Authoring, Board/Graph, Data Sheet, Audit/Validation, Scene Authoring, Asset Production, Appearance/Colour/Texture, Audio, Debug, UI/Input Feedback, and Content Generation.	Extensions own concrete workflows and may depend on Core. They must not assume unrelated extensions exist.
Full Bundle package	Installs all extensions and enables richer cross-lab dashboards, bridges, handoffs, context menus, and coordinated workflows.	Bundle success is not Standalone sufficient. configurations must still compile and run.
Optional bridge packages	Connect two labs or a lab and a third-party package.	Bridges must remain removable and gracefully degraded when absent.
Project adapters	Thin game-specific wrappers outside the generic package.	They may reference project classes. Generic packages must not.

Definition of a PungentFunk utility

- Solves a broadly reusable Unity workflow problem.
- Uses generic names, generic data models, stable IDs, and configurable presets.
- Works in a clean project or clearly declares optional dependencies.
- Provides preview or dry-run before destructive changes and records Undo where scene or asset changes are made.
- Uses shared visual language and persistence helpers unless deliberately exempt.
- Chooses an appropriate Unity Editor integration point rather than forcing every workflow into one window.
- Remains visible, reachable, usable, and verified after refactors and cleanup passes.
- Declares package ownership, dependencies, capabilities, extension points, bridge state, and missing-provider fallback
behavior.

2. Product Architecture

Package layers

Layer	Purpose	Rules
Core	Shared shell, registry, control panel, lab menus, theme, preferences, package metadata, shared scan/result models, documentation patterns, design tokens, provider registries, and Authoring Data Foundation contracts.	No Core-to-lab dependencies. Keep public APIs conservative. Core may own interfaces and fallback UI, not lab workflows.
Runtime model assemblies	ScriptableObjects, MonoBehaviours, data models, adapters, and runtime services used outside the Unity Editor.	No UnityEditor references. No editor-only menu logic. Use stable IDs, version fields, and migration-safe data.
Editor lab assemblies	Editor windows, inspectors, scanners, generators, validators, TreeViews, Scene GUI helpers, overlays, graph tools, and authoring panels.	May depend on UnityEditor and Core. Scan/apply operations must be explicit, cancelable where expensive, and cached. Drawing code should not own mutation logic.
Optional bridges	Code connecting labs or third-party packages.	Use version defines, reflection-safe discovery, asmdef constraints, isolated bridge packages, package

metadata, or discovery.		registry/service
Project adapters packages must never	Game-specific wrappers and presets outside generic packages.	Thin only. Generic compile against them.

Dependency direction

Allowed	Discouraged	Forbidden
Lab -> Core; Editor -> Runtime; Project Editor; Generic Adapter -> Generic Lab; Bridge -> Lab A + class; cyclic lab Lab B compile-time labs.	Lab A directly compiling against Lab B unless Lab B is declared required for that product.	Core -> Lab; Runtime -> Lab -> game-specific dependencies; accidental dependencies on optional

Shared abstraction rule

A helper belongs in Core only when at least two labs need it and the API is stable enough to support publicly. Immature helpers should stay local until repeated need proves they are truly shared.

Core Anti-Sink Rule

Core exists to reduce duplication, not to absorb product features. Core may own stable contracts, service lookup, shared state vocabulary, provider registries, descriptors, fallback states, and simple shell UI. Core must not own concrete lab editors, domain-specific scanner logic, rich document editing, board editing, sheet editing, audit provider implementations, scene placement logic, colour/audio/debug workflows, or package-specific generation behavior.

Decision test

If the code interprets domain-specific meaning, generates domain-specific assets, scans domain-specific content, or renders a concrete editor workflow, it belongs in a lab or bridge. If it only defines stable identity, metadata, provider contracts, shared UI/service vocabulary, or safe fallback state, it can belong in Core.

3. Package Modularity and Extension Ownership Doctrine

PungentFunk Utilities must support modular optional extension packages. The free package should be useful and stable, but it should not force every advanced editor or lab workflow into Core. The full bundle should install and coordinate all extensions, but no extension should rely on the full bundle being present unless it explicitly declares itself bundle-only.

Distribution tiers

Tier	Owns	Must not own
Free/Core	Stable contracts, registries, shared theme/preferences, minimal browser shells, shared scan and authoring contracts.	Concrete lab workflows, advanced editors, or paid-domain implementations.
Paid extension	Concrete lab UI, workflows, domain models, scanners, generators, import/export, templates, and panels.	Core-only services or direct dependencies on unrelated optional packages.
Bridge	Cross-package handoffs and optional integration glue.	Primary ownership of either side workflow.
Full Bundle	Packaging, samples, all bridge enablement, bundle landing pages, cross-lab dashboards.	Hidden dependency assumptions that make standalone packages fragile.
Project adapter	Game-specific presets, wrappers, and compatibility aliases.	Generic package logic or assumptions.

Package capability declarations

Every significant utility, provider, bridge, and authoring item provider should declare package ownership and capabilities.
This allows the Utilities Browser, Authoring Browser, Help Browser, Developer Mode filters, and optional bridge UI to explain what is installed, what is missing, and which package provides each capability.

- packageId and packageDisplayName
- packageTier: Core, Extension, Bridge, Bundle, ProjectAdapter, Internal
- packageOwner and moduleId
- requiredPackageIds and optionalPackageIds
- providedCapabilities and consumedCapabilities
- extensionPointsProvided and extensionPointsConsumed
- bridgeId and bridgeAvailabilityState
- fallbackBehavior and missingDependencyMessage
- installHint and minimumCompatibleVersion
- isDeveloperOnly, isExperimental, and isBundleOnly
- documentationTopicId and relatedUtilityIds

Bridge availability states

- InstalledAndEnabled
- InstalledButDisabled
- MissingRequiredPackage
- VersionMismatch
- MissingOptionalProvider
- DeveloperModeOnly
- UnsupportedInThisContext

Extension feature gating

Feature gates should be visible, calm, and explanatory. A missing optional package must not compile-break, throw runtime/editor exceptions, leave dead buttons, or hide data. Unsupported actions should be disabled with a concise reason and, where appropriate, an install or enablement hint.

Future Codex package split rule

Every Codex brief that adds a utility, provider, editor, scanner, data type, bridge, or menu entry must state which package owns it, which package dependencies are required, which dependencies are optional, which capabilities it provides, which extension points it consumes, and how the UI behaves when optional packages are missing.

4. Shared Authoring Data Foundation Doctrine

The Authoring Data Foundation is a Core-owned shared model/contracts layer. It allows notes, rich documents, board documents, data sheets, tasks, help topics, token definitions, documentation links, audit issues, utilities, and external targets to share IDs, references, metadata, preview actions, open/copy/ping behavior, and validation results without forcing all editor packages to depend on each other.

Browser versus editor surface separation

Surface	Owns	Does not own
Authoring Browser	Browse, filter, search, metadata, backlinks, documents, boards, or context links, read-only previews, provider status, and launch actions.	Full editing of rich sheets.
Rich Document Editor	Text authoring, document structure, inline registry, board editing, or token insertion, templates, document-specific commands.	Global authoring table editing.
Board / Whiteboard Editor	Spatial cards, nodes, relationships, editing or CSV/table dependency maps, canvases, graph minimaps.	Long-form prose workflows.
Spreadsheet / Data Sheet Editor	Tables, CSV, scanner-generated sheets, or graph layout. token/content/data coverage, structured data editing.	Rich prose editing
Shared Authoring Foundation	IDs, link targets, metadata, or domain-specific preview/action/validation contracts, migration fields, provider registration.	Concrete editor UI workflows.

Core-owned model scope

- Stable authoring ID helper.
- Authoring item type enum or registry: LegacyNote, RichDocument, BoardDocument, DataSheet, Task, HelpTopic, TokenDefinition, DocumentationLink, AuditIssue, Utility, ExternalTarget.
- Authoring link target model: asset GUID, local file path, web URL, utility ID, future utility ID, help topic ID, token key, note/document ID, board ID, sheet ID, audit issue code, script path, component type, serialized property path, scene object global ID, and external path.
- Shared metadata: title, summary, tags, status, priority, visibility, created/updated timestamps, source, package owner, extension owner, stable key, archived flag, developer-only flag, and migration version.
- Shared open/copy/ping/action contract.
- Shared preview provider contract.
- Shared validation result contract.
- Shared migration/version fields.
- Optional provider registration for document, board, sheet, note, token, help, documentation-link, audit, utility, and external target providers.
- Safe missing-provider behavior.

Provider contracts

- IPungentAuthoringItemProvider
- IPungentAuthoringPreviewProvider
- IPungentAuthoringActionProvider
- IPungentAuthoringValidationProvider
- IPungentAuthoringSearchProvider
- IPungentAuthoringMigrationProvider

Safe missing-provider behavior

- Keep the item visible in the browser.
- Preserve metadata, source package, stable ID, tags, backlinks, and summary.
- Show a fallback preview if available.
- Disable unsupported editor actions with a clear reason.
- Offer Copy ID, Copy path/link, Ping asset/path, Open raw asset/file, or Create linked note where safe.
- Show which package provides the missing capability.
- Never remove, corrupt, or silently hide records because a concrete editor package is absent.

Legacy Notes and Roadmap migration doctrine

Do not remove the Notes system. Preserve the robust browser, automation, metadata, context menu, inspector, scene, audit, token, help, documentation-link, and utility integration features. Deprecate only the old freeform body editing panel inside Notes and Roadmap.

- Keep legacy notes browseable and manageable records.
- Expose PungentNote records through a LegacyNote authoring provider.
- Map existing note targets into shared authoring link targets.
- Show the legacy body as read-only preview/summary in the Authoring Browser.
- Route real editing to Rich Document Editor when installed.
- When Rich Document Editor is absent, show read-only preview plus copy/export/missing package actions.
- Preserve stable IDs, backlinks, context menus, inspector summaries, scene badges, help bridges, audit bridges, token links, and documentation-link relationships.
- Migrate legacy note bodies by linked copy first. Do not destructively convert or delete PungentNote.body.

Concurrency rule for documents, boards, and sheets

Rich Document Editor, Board/Whiteboard Editor, and Data Sheet Editor may proceed concurrently only after the Authoring Data Foundation contracts compile and the Legacy Note provider demonstrates the shared ID/link/metadata/preview/action model. No track may invent its own permanent ID, link, metadata, action, or validation model unless it first proves the shared foundation cannot represent the need.

5. Source-Truth and Verifiability Doctrine

This section prevents cleanup summaries from being treated as implementation evidence. A debrief is useful context, but source reality and active UI reality win.

Current files outrank all debriefs

A source archive or repository checkout is the source of truth for implementation state. Debriefs, historical briefs, and conversation summaries describe intent, plan, or claimed outcome. They do not prove active implementation. Agents and reviewers must distinguish stated intent, claimed outcome, source reality, and active UI reality.

Evidence levels

Evidence	Acceptable for	Not sufficient for
User current instruction	Intent, priority, override.	Claiming implementation completed.
Current source file	Source implementation.	UI visibility or usability.
Active UI screenshot/manual Unity test	UI reachability and layout.	Code architecture correctness alone.
Snapshot audit	Package state at audit time.	Current state after newer archive.
Debrief or summary	Rationale and claimed goals.	Implementation verification.
Historical chat	Context and motivation.	Current status.

No invisible implementation rule

A feature is not functionally implemented if it only exists in code. UI-facing utility work requires a current source path, active call path, registered access surface where relevant, visible and reachable UI location, usable controls at representative sizes, validation of the intended interaction, and no contradictory duplicate stale version elsewhere.

Claim evidence format

- Feature/control.
- Previous location, if known.
- Current source owner: file, class, method, and state owner.
- Current active UI path: menu, window, tab, panel, foldout, visibility condition.
- Reachability condition.
- Validation performed.
- Known caveat.

Active call path requirement

9. Menu or launcher entry opens the expected window.

10. Window state selects the expected tab/panel/module.

11. The active draw/create method includes the expected section.

12. The section uses current data/state, not stale duplicates.

13. The control is visible and interactable under normal conditions.

14. The action produces or changes the expected state.

6. UI Preservation and No-Hiding Doctrine

UI refactors must improve organization without removing the user ability to perform the workflow. Hiding is not the same as simplifying.

The no-hiding rule

Do not solve a UI issue by hiding functionality users still need. A feature may be collapsed, moved, filtered, or placed behind Developer Mode only when the new access path is intentional, documented, discoverable, and appropriate to its audience.

UI relocation contract

- Before map: list each existing visible control/section and where it lives.

- After map: list the new location and access path.
- Rationale: explain why the new location better matches user workflow.
- Redirect: provide related-tool link, inline handoff, or clear label if the old place no longer hosts it.
- Visibility: ensure the new location is visible at practical window sizes.
- Persistence: preserve relevant preferences, splitter widths, selected tab, search text, filters, and foldouts.
- Validation: test the moved control after compile/domain reload.

UI control inventory gate
Before any cleanup pass on a window, create a control inventory covering entry point, tabs/modules/foldouts, visible sections, actionable controls, filters/search controls, dangerous actions, diagnostic/status panels, developer-only controls,
documentation/help links, and optional integration states. After the pass, every control must be preserved, moved with path, replaced by equivalent, intentionally removed with approved rationale, or gated behind a documented mode.

Duplicate and stale UI rule

Duplicated UI can be worse than missing UI. If two panels appear to expose the same feature but only one is active or current, remove or clearly mark stale panels, keep compatibility aliases only at access-surface level, route old entries to the
new active implementation, avoid identical labels for different behavior, and include a deprecation warning only when the
old path still exists.

Cramped UI is not completed UI
A feature does not count as restored if controls exist but are squeezed into unreadable columns, clipped labels, tiny hit
targets, or scroll traps. Validate narrow width, comfortable width, tall/short heights, docked/floating mode, foldout/tab states,
and empty/partial/populated data states.

7. Editor UI and UX Principles

The suite should have a cohesive visual identity, but not a rigid one. Consistency should come from shared theme colours,
typography rules, spacing scale, button/status/box styles, icons, interaction conventions, safety patterns, registry metadata,
lab navigation, and reusable widgets. Consistency should not mean every workflow is forced into the same vertical inspector layout.

Unity-native alignment
- Buttons trigger actions.
- Toggles enable states.
- Dropdowns choose one option.
- Radio groups choose mutually exclusive modes.
- Sliders adjust continuous ranges.
- Numeric fields set exact values.
- Object fields reference assets or objects.
- Search fields filter.
- Tabs separate workflows.
- Toolbars collect compact actions.
- Progress bars report long work.
- Help boxes explain state.

Responsive layout rules
- The window must remain usable at small sizes.
- Prefer vertical scrolling over horizontal scrolling.
- Use wrapping rows, compact cards, collapsible sections, tabs, paging, and column stacking before horizontal scrolling.
- Use draggable splitters where panel proportions matter, and persist sizes.
- Avoid dead space, jumpy controls, and important actions buried below large scroll regions.
- Responsive means key controls remain discoverable and practical, not merely that the UI technically draws.

Visual information design
- Use UtilityWindowTheme for shared IMGUI chrome.
- Use equivalent design tokens for UI Toolkit tools.
- Use colour intentionally for status, category, severity, selection, lab identity, or risk.
- Do not use colour as the only differentiator.
- Use icons, tags, badges, count chips, legends, and labels for repeated states.
- Use diagrams, mini graphs, swatch grids, matrices, timelines, heatmaps, and graph canvases only where they clarify faster
than text.

8. Window and Panel Layout Doctrine

Each window and panel should be designed deliberately. A utility layout is part of the product design, not decoration.

Layout templates

Template	Best for	Doctrine
Simple vertical stack	Small settings and single-purpose helpers.	Header -> configuration -> action -> dense or workflows.
Two-column layout	Source/result, configuration/preview, selection/detail.	Use persistent draggable splitters where both sides compete for space.
Three-panel layout	Advanced authoring, asset browsers, debug tools, lab hubs.	Left navigation/source, centre workspace/results, right details/actions.

		Collapse or stack at narrow sizes.
Grid/card layout	Asset sets, palettes, icons, prefab previews, generated names, tool launchers.	Cards should wrap and use compact metadata.
Table/matrix layout	Coverage tools, validation, dependency maps, compatibility checks.	Use sorting, filtering, grouping, and detail panes. Horizontal scroll is acceptable when preserving column relationships matters.
Canvas/lab workspace	Generation, placement, spatial workflows, graph-like relationships.	Central workspace, side configuration, contextual toolbar, mode-specific controls.
Wizard/step flow	Setup, migration, export, risky multi-stage operations.	Show step, inputs, validation, dry-run results, and final apply/export.
Dashboard/overview	Control panel, lab home, health summaries.	Surface summaries, warnings, search, recent tools, quick actions. Do not turn dashboards into dense control panels.
Contextual mini panel	Component, scene selection, terrain, or asset quick actions.	Keep compact and include an Open Full Tool handoff for complex work.

Overlay Tray and Popup Window Doctrine

Overlay trays should be the preferred surface for transient popup-style interactions when the task is contextual, lightweight, dismissible, and related to a currently visible utility, browser, panel, tray, or selection. Prefer a shared overlay tray over an ad hoc detached popup EditorWindow, UtilityWindow, floating mini-window, or modal dialog whenever the user benefits from staying in the current workflow.

Use overlay trays for:
- Contextual help, glossary, and documentation previews that need an Open Full Help Browser handoff.
- Minimized utility controls, recent tools, quick actions, and utility switchers.
- Lightweight settings, filters, sort controls, package state explanations, and missing dependency notices.
- Selected-item summaries, backlinks, related utility lists, authoring previews, and bridge/action menus.
- Small pickers, inspectors, confirmation previews, and temporary palettes that support an active panel.

Use a full EditorWindow instead when the workflow needs a persistent dockable workspace, large result review, dense data tables, graph/canvas editing, rich document editing, sheet editing, scene authoring, multi-stage scan/apply flows, or long-lived authoring state. Use a modal dialog only for destructive confirmation, blocking error recovery, required save/discard decisions, or security/safety choices where the user must answer before continuing.

Overlay tray behavior rules:

- A tray must have a clear owner: utility window, panel, toolbar button, selection, scene overlay, minimizer strip, or browser row.
- A tray should be non-modal by default, easy to dismiss, and should never trap the user unless the task is intentionally blocking.
- A tray must clamp within the usable editor/window bounds, handle narrow and short sizes, and avoid covering the primary action that opened it unless no better placement exists.
- User-movable or resizable trays must persist and safely clamp their position, size, collapsed state, and selected topic after domain reload, docking changes, and monitor changes.
- Trays should use shared theme tokens, calm borders, readable elevation/shadow, clear title/action rows, and visible focus states.
- Trays must preserve keyboard and mouse focus predictably. Escape should close dismissible trays. Clicking outside may close lightweight trays unless pinned.
- Trays that contain more than quick content should include Open Full Tool, Open Full Browser, or Dock as Window handoffs.
- A tray must render cached state and invoke explicit actions; it must not own project scans, heavy reflection, asset mutation, scene mutation, or broad document generation from repaint.
- Multiple trays should be coordinated by a shared tray manager or rail. Avoid overlapping floating-popup clutter, duplicate tray stacks, and hidden tray state with no visible affordance.

Overlay tray implementation review gate:

- State why the workflow is tray-appropriate rather than a full EditorWindow, inspector, scene overlay, context menu, or modal dialog.
- Identify the owner surface and active UI path.
- Verify narrow/wide, short/tall, docked/floating, pinned/unpinned, outside-click, Escape, domain reload, and missing-provider states.
- Confirm the tray does not hide essential controls, does not break keyboard focus, and provides a full-tool handoff when the content outgrows the tray.

Layout anti-patterns

- Long undifferentiated walls of fields.
- Excessive nested foldouts.
- Large empty gaps from rigid sizing.
- Avoidable horizontal scrollbars.
- Controls that jump during refresh.
- Important actions hidden below large scroll regions.
- Duplicate buttons doing the same thing.
- Making every utility look identical at the cost of usability.
- Using graph/canvas/wizard UI only because it looks advanced.
- Claiming a section was restored when it is only present in code but not reachable in active UI.

Shared resizable panel contract

- Handle orientation matches motion.
- Drag direction is intuitive.
- Handle placement matches ownership.
- Panel edges stick to handles.
- The remaining panel absorbs remaining space.
- Bounds are relative, not arbitrary.

- Sizes are clamped before drawing.
- Handles remain discoverable.
- Handles do not steal layout space silently.
- No inverted border regression.

Canonical split calculation rule

15. Reserve fixed outer padding, inner spacing, and handle width/height.

16. Compute minimum sizes for both sides and maximum size from current available area.

17. Clamp saved split value before drawing.

18. Draw the controlled panel at clamped size.

19. Draw the handle immediately after that panel.

20. Draw the remaining panel with expansion so it consumes leftover space.

Panel stretching and anchoring rules

- Navigation/source rails usually have clamped width and full height.
- Main workspaces/results panels expand in both directions.
- Inspector/detail rails may have clamped width but stretch vertically.
- Results, previews, logs, documentation, and dense lists receive leftover height.
- Footer/action bars remain visible and are not pushed below unrelated scroll regions.
- Use one primary scroll region per panel unless inner scroll has a distinct data-navigation purpose.

Adaptive row wrapping contract

- Conceptual rows remain together when there is room.
- Semantic rows stay separate.
- Rows wrap by whole control groups, not by splitting labels from fields.
- Each control group has minimum, preferred, and maximum width.
- Wrapped rows preserve reading order and primary action discoverability.
- Horizontal scrollbars are reserved for inherently horizontal artifacts.

Common UI implementation failure matrix

Failure	Symptom	Prevention doctrine
Inverted resize handle ownership naming and validation.	Dragging visually shrinks the panel it appears to own.	Require handle drag-direction
Missing resize handle	Panels compete for space but cannot be adjusted.	Meaningful split layouts need visible, discoverable handles.
Unconstrained panel current available size and every layout pass.	Saved width/height crushes sibling panels after resize/reload.	Clamp against sibling minimums
Detached panel edge handles in strict visual flexible panel absorb	Free space appears between panel and handle.	Draw panels and order and let remainder.
Non-stretching results main workspace/result/detail region.	Results/docs/previews occupy preferred height while unused space remains.	Assign expansion to
Over-crowded toolbar groups with widths.	Buttons, chips, search fields, and help icons clip labels.	Use adaptive row min/preferred/max
Dead helper syndrome path proof for UI	Correct-looking helper exists but active UI path does not call it.	Require active call claims.
Core-to-lab dependency leak abstractions only; lab code bridge assemblies.	Core references Notes, Project Audit, Debug, Palette Designer, or other labs directly.	Core may hold belongs in lab or
Overlay tray misuse overlay trays; use full windows only for	Lightweight popup content opens as a persistent, dense, or multi-stage workflows. detached window/modal and interrupts the current workflow.	Prefer shared
Repaint scanning only; run scans through staged services.	Indexing/reflection/doc generation runs from OnGUI/draw paths.	Draw current state explicit cached

9. Progressive Disclosure, Assistance, Accessibility, and Messaging

Progressive disclosure rules

- Show the core workflow first.
- Collapse advanced options by default only if the core workflow remains complete.
- Do not bury essential controls under advanced sections.
- Do not move required controls into Developer Mode unless they are truly developer-only.
- If a feature is filtered out, provide visible filter state and a way to reveal it.
- If a feature is archived or hidden, provide restore or show-hidden controls where appropriate.

Tooltips and contextual help

- Every meaningful field should have a tooltip or help affordance.
- Tooltips explain what the field controls, when to change it, how it differs from related options, and whether it is destructive, experimental, expensive, or preview-only.
- Complex enums should have nearby descriptions.
- Labels should describe the enabled state or action clearly.
- Help affordances should preserve context: utility ID, package ID, tab/module, section, selected item, and topic anchor.

Accessibility expectations

- Do not rely on colour alone.
- Pair colour with icons, labels, shapes, patterns, or tooltips.
- Maintain readable contrast.
- Provide visible focus states where feasible.
- Preserve logical keyboard navigation where supported.
- Avoid tiny click targets for frequent actions.
- Use plain-language labels for common operations.
- Runtime UI and Feedback Lab components should expose meaningful accessibility hierarchy, labels, roles, values, and state where supported.

Empty states and errors

Empty states should explain what is missing and offer one clear next action. They should not use error styling unless something failed. Errors state the problem, likely cause, and next useful step. Warnings identify risk before changes are applied. Info messages confirm state without blocking workflow.

10. Unity Editor Implementation Doctrine

PungentFunk tools should choose Unity Editor implementation patterns deliberately. This is a selection framework, not a mandate to rewrite stable tools.

IMGUI, UI Toolkit, and migration policy

- IMGUI remains acceptable for existing windows, small utility panels, debug tools, compatibility wrappers, and internal workflows where code-driven rendering is fastest and clearest.
- UI Toolkit should be evaluated first for new flagship lab windows, persistent complex workspaces, product-facing tools, rich styling, component-rich views, retained state, UXML/USS-driven presentation, and tools expected to grow significantly.
- Do not rewrite for fashion. Rewrite when there is a concrete product, maintainability, accessibility, performance, or UX reason.
- Hybrid strategies are acceptable.
- Major tools should document why they use IMGUI, UI Toolkit, CustomEditor, Overlay, TreeView, graph/canvas, or a hybrid pattern.

Implementation role selection

Role	Use when	Doctrine
EditorWindow / Lab Window	Workflow has filters, previews, generated results, scan/apply stages, dashboards, tabs, or persistent workspace.	Keep scan/apply logic outside repaint. Cache expensive state.
CustomEditor	Tool belongs directly to a selected component or asset.	Keep compact. Provide validation, setup helpers, and Open in Lab handoffs.
PropertyDrawer	A serializable field type needs reusable presentation.	Use for field-level UX, not whole workflows.
Scene GUI / Handles	User edits spatial data directly in Scene View.	Keep lightweight. Respect Undo, snapping, visibility, and Prefab Mode.
Scene View Overlay / EditorTool	Persistent compact controls are needed while working in Scene View.	Use for placement, navigation, debug toggles, paths, terrain/surface, and environment previews.
TreeView / MultiColumnView	Data is hierarchical, dense, expandable, or audit-like.	Preserve row identity, selection, expansion, sorting, filtering, and scroll position.
Search Provider / Query Tool	Dataset is large and users think in terms of discovery/filtering.	Avoid inventing custom query syntax unless the lab needs it.
Shortcut / Command	Action is frequent, safe, contextual, and easy to describe.	Avoid rare or destructive shortcuts.

Context Menu	Short action is most useful from selection.	Keep concise and route full workflows to labs.
Graph / Canvas Tool	Relationships, branching, dependencies, or pipelines are the primary authoring problem.	Use canvas only where it clarifies more than forms, lists, or tables.
Optional Bridge	Feature links labs or optional packages.	Keep removable and gracefully degraded.

11. Safety and Performance Doctrine

Drawing must not own mutation

Editor drawing methods should render current state. They should not own expensive scans, asset writes, scene mutation, graph mutation, broad reflection work, project-wide operations, or background task scheduling.

Preview/apply flow

21. Configure.

22. Scan or validate.

23. Preview.

24. Apply selected or apply all.

25. Summarize changed, skipped, and failed items.

Performance standards

- Do not scan all scenes or assets on every visual refresh.
- Cache scan results and rebuild only when settings change or the user refreshes.
- Use staged editor work for long preview, export, bake, icon, or scan operations.
- Use cancelable progress for expensive work where practical.
- Avoid Thread.Sleep in editor tools.
- Avoid broad SceneView repaint unless required.
- Preserve row/node identity in large lists, trees, tables, and graphs.
- Use virtualization/pooling for large UI Toolkit lists.
- Static state is a cache, not source-of-truth user data.

Safety standards

- Preview or dry-run before destructive and batch operations.
- Default dangerous toggles to safe behavior.
- Do not overwrite user data without explicit consent.
- Do not modify shared materials, prefab assets, imported assets, or scenes by default without warning.
- Show a summary of changed, ignored, and failed items.
- Be explicit about scope: selection, active scene, open scenes, prefab assets, project assets, generated groups, or all matching files.

12. Shared Package Systems and Project Scan Pipeline Doctrine

Project-wide scanning is a package-level service, not a per-window implementation detail. Future panels that need broad project facts should consume the shared scan pipeline, cache, provider registry, project audit index, and shared findings UI before creating local scanners.

Shared systems first rule

Before creating a new registry, scanner, help link system, note link system, token parser, package visibility model, progress tracker, or validation result type, check whether an existing package-wide service or contract can be consumed or extended.

Shared service ownership model

Owner	Owns	Does not own
Core	Contracts, result models, provider registries, caches, coordination policies, fallback UI states.	Domain-specific scanners or editor workflows.
Labs/extensions	Domain providers, concrete result interpretation, apply/fix actions, specialist UI.	Generic scan orchestration duplicated locally.
Bridges	Cross-lab result transformations and handoffs.	Primary scan ownership.
Project adapters	Game-specific providers and wrappers.	Generic package rules.

Scan ownership model

Core may own scan contracts, result models, severity vocabulary, provider registration, project audit cache/index concepts, cooperative scan coordination, progress/state presentation helpers, and background/idle scheduling policy. Domain-specific scan providers belong in labs, bridges, or project adapters. Windows and panels consume scan state; they should not duplicate broad scan orchestration.

When to register a scan provider

- The scan produces reusable project facts, findings, coverage, dependencies, token data, documentation state, package metadata, or release readiness state.
- Another utility could reasonably consume the results.

- The scan may become expensive enough to need scheduling, cancellation, stale-state reporting, or caching.
- The scan should appear in package-wide summaries or findings UI.

Local scan session versus project audit provider
A local scan session is acceptable for a narrow one-off operation owned by a single tool. A project audit provider is required when results are reusable, global, cacheable, or useful to other packages. If uncertain, start with a local scan session whose result shape can be promoted into a provider later.

Shared scan result contract
- Provider ID and package owner.
- Scan scope and source.
- Timestamp, duration, cache freshness, stale state.
- Severity, category, title, summary, detail, target references.
- Suggested actions, fix availability, dry-run support, and validation links.
- Stable finding IDs where possible.
- Optional sheet/export/authoring link metadata.

Background and immediate scan doctrine
Background scans should be cooperative, idle-aware, pause/cancel aware, and unobtrusive. Immediate scans may be faster and more intrusive, but should be explicitly user-triggered and should still report progress, cancellation, and stale cache state where possible.

Cache and stale-state doctrine
Scan-consuming windows should show cached results on open, communicate last scan age and stale state, avoid forced rescans unless explicitly requested or safely scheduled, and provide refresh controls that state their scope and expected cost.

Scan UI doctrine
Findings UI must be readable, grouped, filterable, and action-oriented. Dense findings cards should surface severity, title, target, package/provider, freshness, and the most useful action first. Details, raw identifiers, and developer-only diagnostics should be progressively disclosed.

Repaint safety for shared services
No scan, indexing, reflection, script parsing, documentation generation, or broad project query should run from routine repaint. Drawing code draws cached state. Services produce state through explicit actions, staged editor work, background tasks, or controlled provider refreshes.

13. Editor Access and Integration Principles

Menu items are the baseline access method, not the only access method. Tools should appear where the user naturally needs them.

Access method	Principle
Organised menus	Use clean, curated menu roots. Avoid duplicate clutter and old project-specific names except as compatibility aliases.
Utility Browser / Lab Hub	Every significant utility should register with the central registry and be searchable/filterable.
Toolbar / command access	Use sparingly for frequent global or status-like actions. Do not mirror the full menu tree.
Scene View overlays	Use for placement, surface alignment, navigation, gizmo browsing, debug visual toggles, path tools, terrain/surface helpers, and environment previews.
Context menus	Use for concise selected-object, asset, folder, material, prefab, texture, terrain, palette, or ScriptableObject actions.
Inspector integration	Use compact toolbars, validation panels, quick actions, dependency messages, and Open in Lab buttons.
Cross-utility embedding	Use optional services, registry lookup, package detection, reflection-safe adapters, or bridge packages.
Shortcuts	Use for frequent, safe, easy-to-describe commands.
Context-aware launch	Preload relevant selection context and fall back gracefully.
Recent/pinned/resume	Preserve recent tools, favourite labs, last active module, and related-tool handoffs through shared preferences.

Access-surface preservation
When moving or consolidating access paths, preserve discoverability. Legacy aliases may route to the new active tool, but should not maintain competing stale implementations. If a utility becomes package-gated, the access surface should explain which package owns it and what functionality remains available.

14. Registry, Navigation, and Lab Hubs

The registry is the package-level discovery contract. Every significant utility should have complete metadata, package ownership, capability declarations, and optional dependency information.

Field	Purpose
Id	Stable machine-readable kebab-case identifier. Never reuse for a different tool.
DisplayName	User-facing, generic, clear, asset-store friendly name.
Lab / Module / Category	Product area, subdomain, and search/menu taxonomy.
Description	One-sentence promise: what the tool does and when to use it.
PackageStatus	Stable, Experimental, In Progress, Deprecated, Project Adapter.
ImplementationRole	EditorWindow, CustomEditor, PropertyDrawer, TreeView, Scene GUI, Overlay, Runtime Service, Bridge, Graph Tool, or Hybrid.
RelatedUtilityIds	Complementary tool links. Must not imply hard dependency unless declared.
CapabilityFlags	Selection, context menu, scene overlay, batch action, lab hub, tree data, scene handles, inspector embedding, graph workspace, search, shortcut, accessibility metadata, package bridge.
Package metadata	packageId, packageTier, required/optional package IDs, provided/consumed capabilities, extension points, bridge availability, missing dependency message, documentation topic.

Menu policy
- Primary editor menus: Tools/PungentFunk Utilities/...
- Asset context menus: Assets/PungentFunk Utilities/...
- GameObject context menus: GameObject/PungentFunk Utilities/...
- CreateAssetMenu root: PungentFunk Utilities/[Lab]/[Asset Type]
- Legacy aliases are allowed only as compatibility routes and should be marked internally.

Visibility model rules
Developer/internal/archived utilities should be visible through explicit Developer Mode filters, not hidden behind obscure conditions. Missing package states should appear as disabled or informational cards where discovery is useful.

15. Optional Dependency and Cross-Utility Integration Principles

PungentFunk Utilities should become more powerful when multiple labs are installed together without making standalone labs fragile.

Dependency levels

Level	Meaning
Required dependency	Must exist for the package to compile and function. Keep minimal and documented.
Optional integration	Activates when another lab or package is installed. Must not compile-break when absent.
Suggested companion	Improves workflow but is not required. Missing companion messaging should be calm and non-blocking.
Bridge dependency	Isolated package or asmdef that can reference both sides and be removed without breaking either side.

Optional integration techniques
- Assembly definition optional references.
- Version defines.
- Define constraints.
- Reflection-safe discovery.
- Interface packages.
- Adapter classes in separate integration folders.
- Registry-based service lookup.
- Loose ScriptableObject references.
- Conditional compilation.
- Soft dependency metadata in descriptors.
- Disabled-but-helpful UI state when absent.

Cross-lab dependency rule

Cross-lab dependencies are product decisions, not accidental using directives. If a feature links two labs, prefer a bridge.
Direct lab-to-lab compile dependencies are acceptable only when the dependency is required and documented for that product package.

16. Runtime, Editor, Data, and Serialization Rules

Runtime package rules
- Runtime scripts must compile in player builds without UnityEditor.
- Runtime ScriptableObjects should contain data and pure runtime logic only.
- Do not store editor-window state in runtime assets unless it has runtime meaning.
- Serialized fields should have tooltips and sane defaults.

- Use stable IDs or GUID-like keys for references across assets, adapters, graphs, packages, and authoring items.

Editor package rules
- Editor windows should use SerializedObject and SerializedProperty when editing serialized assets/components where Undo,
  prefab overrides, or multi-object editing matters.
- Use Undo for scene/object changes.
- Use AssetDatabase and EditorUtility.SetDirty safely.
- Prompt before opening/changing scenes, editing prefab assets, or applying destructive project-wide changes.
- Separate rendering from scanning, asset mutation, scene mutation, graph mutation, and service logic.

Serialization and prefab doctrine
Prefab-facing tools must distinguish prefab assets, prefab instances, variants, nested prefabs, Prefab Mode isolation, and
in-context prefab editing. Do not silently apply scene-instance choices to prefab assets. Generated assets should preserve
GUID/meta stability where possible and avoid delete/recreate churn when update-in-place is safe.

Authoring data serialization doctrine
Authoring item IDs and link targets must be stable across package installs, domain reloads, asset moves, and optional
package availability changes. Missing providers should not delete or rewrite persisted authoring records. Migration/version
fields should be explicit and forward-compatible.

17. Asset, Package, and Import Pipeline Doctrine

Package and distribution doctrine
- Design labs so they can ship as standalone products and as a bundle.
- Use package boundaries, asmdefs, samples, documentation, icons, presets, and optional bridges deliberately.
- Keep content organised: runtime code, editor code, samples, documentation, icons, presets, migration helpers.
- Do not rely on project-specific folder paths.
- Default output folders should be configurable, safe, and visible.
- Use special Unity folders deliberately and sparingly.

AssetDatabase and import workflow doctrine
- AssetDatabase is editor-only.
- Asset operations should be explicit and previewed when they affect many assets.
- Batch operations should avoid unnecessary refresh/import churn.
- Tools should distinguish source assets, generated assets, temporary previews, and user-authored assets.
- Generated asset names should be deterministic where useful, collision-safe, and reversible.
- Generated docs must not overwrite curated docs unless the user explicitly requests replacement.

18. Package Split Map

This package map defines the recommended ownership split for release planning. It is doctrine for future briefs and implementation passes until superseded by a formal package manifest.

18.1 Free / Core package	
Tier	Free/Core.
Required dependencies	Unity Editor baseline only.
Optional dependencies	Any extension package.
Provides	Utility registry/descriptors, utility browser/launcher, shared support, theme/preferences, minimizer/tray, layout helpers, shared contracts/result models/cache shell, shared token parser contracts, Authoring Data Foundation contracts, authoring item IDs, link targets, metadata/status/tag model, validation result model, preview/action contracts, lightweight Authoring Browser core.
Consumes	Optional providers from installed extensions.
Likely utility IDs	utilities-browser, utility-window-theme, minimized-utilities, documentation-links-reader, help-browser-core, authoring-browser-core, project-scan-contracts, token-parser-core.
Release rule	Free/core. Keep useful but not bloated. It may browse and open installed providers, but should not ship all editors.
18.2 Full Bundle package	
Tier	Paid full bundle.
Required dependencies	Core plus all extension packages.
Optional dependencies	Third-party integrations.
Provides	Everything installed; all official bridges enabled; bundle landing page; samples; coordinated dashboards.
Consumes	All package extension points.
Release rule	Bundle convenience only. It must not become the only configuration that compiles or works.
18.3 Documentation and Authoring package	
Tier	Paid extension, with some developer/internal authoring controls gated.
Required dependencies	Core.
Optional dependencies	Audit/Validation, Board/Graph, Data Sheet, Content

Generation,	
	UI/Input Feedback.
Provides	Rich Document Editor, advanced Authoring Browser features, Help Browser authoring extensions, Documentation Links
editing	tools, token/document insertion helpers, game-text
authoring	templates, Twine/Yarn-inspired document templates/flows, implementation brief/release note/debug log templates.
Consumes	Token providers, help providers, documentation providers, authoring item providers.
Likely utility IDs	rich-document-editor, authoring-browser-advanced, help-
browser-	authoring, documentation-links-editor, document-template-
library,	game-text-authoring, brief-template-authoring.
Bridges	Authoring to Audit, Authoring to Token, Authoring to
Content	Generation, Authoring to Board, Authoring to Data Sheet.
Release rule	This is where serious writing/editing belongs. Notes
Browser shell	remains reduced/free in Core.

18.4 Board / Graph / Visualization package

Tier	Paid extension.
Required dependencies	Core.
Optional dependencies	Documentation and Authoring, Audit/Validation, Data Sheet.
Provides	Whiteboard/board documents, configurable node graph
editor,	relationship/dependency map views, minimap/graph widgets, blackboard node data, graph visualization components.
Consumes	Authoring item registry, preview/action contracts,
optional	audit/document/sheet links.
Likely utility IDs	board-document-editor, whiteboard-editor, node-graph-
editor,	relationship-map, dependency-map-view, graph-minimap, blackboard-node-data.
Bridges	Board to Authoring, Board to Audit, Board to Data Sheet.
Release rule	Use shared authoring IDs/links. Do not invent a separate permanent link target system.

18.5 Spreadsheet / Data Sheet package

Tier	Paid extension.
Required dependencies	Core.
Optional dependencies	Audit/Validation, Documentation and Authoring, Token
Validator,	Content Generation.
Provides	Data sheet model, table editor, CSV import/export,
scanner-	generated sheets, SerializedProperty binding/adapters, token/content sheets, Coverage Matrix successor.
Consumes	Authoring item contracts, token contracts, scan
result/finding	models.
Likely utility IDs	data-sheet-editor, csv-import-export, scanner-generated-
sheets,	serialized-property-sheet-adapter, token-content-sheet,
coverage-	matrix-next.
Bridges	Sheet to Audit, Sheet to Token, Sheet to Authoring, Sheet
to	Content Generation.
Release rule	Coverage Matrix should be deprecated into this package
with	compatibility aliases.

18.6 Audit / Validation / Scanning package

Tier	Paid extension; provider-authoring controls may be developer/internal.
Required dependencies	Core.
Optional dependencies	Documentation and Authoring, Data Sheet, Scene Authoring, Audio, Asset Production, Debug.
Provides	Design Validation Audit advanced UI, project audit
providers,	coverage scanners, reference assignment scanner, terrain
usage	scanner, token validator advanced/project scan features,
release	readiness checks, findings actions.
Consumes	Core scan contracts/cache/provider registry, authoring
note/action	contracts, optional sheet export.
Likely utility IDs	design-validation-audit, project-findings, reference-
assignment-	scanner, terrain-usage-scanner, token-validator-advanced, release-readiness-checks, audit-finding-actions.
Bridges	Audit to Authoring, Audit to Data Sheet, Audit to Scene

Authoring,	Audit to Audio.
Release rule	Core owns scan contracts. This package owns advanced scan
UI	and domain providers.

18.7 Scene Authoring / Placement package

Tier	Paid extension.
Required dependencies	Core.
Optional dependencies	Audit/Validation, Board/Graph, Asset Production.
Provides	Asset Placement Lab, Surface Align, Modular Path Builder, Scene Navigation, Scene Gizmo Browser, Area Authoring Toolkit, placement repair/context tools.
Consumes	Core registry/theme/layout, optional audit findings, optional graph visualization.
Likely utility IDs	asset-placement-lab, surface-align-tool, modular-path-builder, scene-navigation, scene-gizmo-browser, area-authoring-toolkit, placement-repair-tools.
Bridges	Scene to Audit, Scene to Board, Scene to Asset Production.

18.8 Asset Production package

Tier	Paid extension.
Required dependencies	Core.
Optional dependencies	Audit/Validation, Appearance/Colour/Texture.
Provides	Prefab Icon Generator, Prefab Asset Exporter, Font Preview, Texture Array Baker, preview/export queues.
Consumes	Core preview/action contracts, optional colour/texture data, optional audit findings.
Likely utility IDs	prefab-icon-generator, prefab-asset-exporter, font-preview, texture-array-baker, asset-preview-queue, asset-export-queue.
Bridges	Asset Production to Audit, Asset Production to Colour/Texture.

18.9 Appearance / Colour / Texture package

Tier	Paid extension.
Required dependencies	Core.
Optional dependencies	Asset Production, Documentation and Authoring, UI/Input Feedback.
Provides	Palette Designer, colour analysis/accessibility tools, theme preview integrations, procedural texture generation where appropriate.
Consumes	Theme contracts, preview providers, optional document/theme references.
Likely utility IDs	palette-designer, colour-analysis, colour-accessibility-preview, theme-preview-integration, texture-generator.
Bridges	Colour to Asset Production, Colour to Documentation, Colour to UI/Input.

18.10 Audio Authoring package

Tier	Paid extension.
Required dependencies	Core.
Optional dependencies	Audit/Validation, Data Sheet.
Provides	Audio Setup Coverage, Audio Catalog Coverage, audio clip/profile/matrix authoring, audio validation providers.
Consumes	Scan contracts and sheet export contracts.
Likely utility IDs	audio-setup-coverage, audio-catalog-coverage, audio-profile-matrix, audio-validation-provider.
Bridges	Audio to Audit, Audio to Data Sheet.

18.11 Debug / Diagnostics package

Tier	Paid extension or developer-focused extension.
Required dependencies	Core.
Optional dependencies	Audit/Validation, Documentation and Authoring.
Provides	Debug Controller, debug router panels, scheduled actions, signal/channel diagnostics, reflected component state panels.
Consumes	Registry, Developer Mode, optional findings/note contracts.
Likely utility IDs	debug-controller, debug-router, scheduled-debug-actions, signal-channel-diagnostics, reflected-component-state-panel.
Bridges	Debug to Audit, Debug to Authoring.

18.12 UI / Input Feedback package

Tier	Paid extension, with possible free runtime subset later.
Required dependencies	Core.
Optional dependencies	Documentation and Authoring, Content Generation,

	Audit/Validation.
Provides	Input Prompt Icon Library, input prompt/token bridges, future UI
Consumes	feedback runtime/editor presenters. Token contracts, documentation links, optional accessibility/validation findings.
Likely utility IDs	input-prompt-icon-library, input-prompt-library-populator, input-
Bridges	token-bridge, ui-feedback-presenters. UI/Input to Token, UI/Input to Authoring, UI/Input to Audit.

18.13 Content Generation package

Tier	Paid extension.
Required dependencies	Core.
Optional dependencies	Documentation and Authoring, Data Sheet, UI/Input Feedback.
Provides	Name Generator, MadLib/name list tools, future text generation
Consumes	helpers, document/content export bridge. Authoring item/action contracts, optional sheet data, optional
Likely utility IDs	token contracts. name-generator, madlib-generator, name-list-tools, text-
Bridges	generation-helpers, content-export-bridge. Content to Authoring, Content to Data Sheet, Content to UI/Input.

19. Lab Taxonomy

Lab	Product role	Primary ownership
Utility Core	Shared shell and UX infrastructure.	Registry, control panel, lab menus, prefs, shared registries, shared Authoring Data tray/minimizer, theme, contracts, service scan/result models, Foundation contracts.
Documentation and Authoring	Focused rich document and documentation authoring.	Rich Document Editor, Authoring Browser, advanced help/docs authoring tools, templates.
Board / Graph / Visualization	Spatial and relationship-heavy authoring.	Board documents, node graphs, relationship canvases. dependency maps,
Spreadsheet / Data Sheet	Structured tabular authoring and coverage data.	Data sheet editor, CSV import/export, Matrix scanner sheets, Coverage successor.
Audit / Validation / Scanning	Project health, validation, release readiness, coverage.	Design Validation Audit advanced UI, actions, advanced providers, findings token scan UI.
Scene Authoring / Placement	Reusable scene authoring workflows.	Placement, alignment, paths, gizmos, area navigation. authoring, scene
Asset Production	Preview, generation, export, and production utilities.	Prefab icons, prefab export, font preview, preview/export queues. texture arrays,
Appearance / Colour / Texture	Colour, palette, theme, visual analysis, procedural appearance support.	Palette Designer, colour accessibility, generation where theme preview, texture appearance-focused.
Audio Authoring	Audio setup, catalog, coverage, and validation.	Audio coverage windows, audio audio validation matrix/profile authoring, providers.
Debug / Diagnostics	Debug control and diagnostic observation.	Debug Controller, routers, scheduled diagnostics. actions, signal/channel
UI / Input Feedback	Prompt/icon libraries and UI feedback surfaces.	Input prompt icon library, input token presenters. bridges, feedback
Content Generation	Reusable content generation helpers.	Name Generator, MadLib/name tools, text

generation helpers, export bridge.

20. Documentation Architecture

Documentation should be layered so humans and AI agents can find the correct source quickly. The design bible stays durable and high-level. Snapshot audits, feature inventories, lab specs, pattern pages, implementation briefs, and skills are
companion documents with clear jobs.

Layer	Artifact	Purpose	Volatility
Vision	docs/design-bible/index.md or vision.md	Short alignment, product promise, labs, reading paths.	Low
Doctrine	docs/design-bible/doctrine.md	Stable architecture, UX, safety, package doctrine.	Low
Patterns	docs/design-bible/patterns/*.md	Concrete reusable implementation patterns.	Medium
Labs	docs/design-bible/labs/*.md	Per-lab scope, dependencies, workflows, definition of done.	Medium
Agent instructions	.github/copilot-instructions.md, .github/instructions/*.instructions.md, .github/skills/.../SKILL.md	AI behavior, repo rules, path-specific rules, task procedures.	Medium
Snapshots	Documentation/Snapshots/*	Script counts, feature inventory, audit reports.	High
Execution	Issues, PRs, Codex briefs, update-pass notes.	Work planning and implementation deltas.	High
Release	CHANGELOG.md, release notes, tagged PDFs.	Human-readable change history.	Medium

Documentation source precedence

- Curated lab docs outrank generated supplements.
- Curated utility notes outrank generated index drafts.
- Generated docs may fill missing sections but must not overwrite curated content without explicit approval.
- Snapshot audits describe implementation state at audit time only.
- Debriefs and historical briefs are rationale, not proof.

21. AI Task Routing and Failure Prevention Matrix

AI agents are expected to treat this bible as a normative decision system, not decorative background reading.

Agent priorities

26. Identify the affected package layer: Core, lab, runtime model, editor tool, bridge, or project adapter.

27. Identify the intended implementation role: EditorWindow, CustomEditor, PropertyDrawer, Scene GUI, Overlay, TreeView,
Search Provider, Shortcut, Runtime Service, Bridge, Graph Tool, or hybrid.

28. Check whether the existing implementation role matches the user workflow.

29. Preserve runtime/editor separation.

30. Keep generic packages free of game-specific references.

31. Keep optional integrations removable.

32. Separate drawing from scanning and mutation.

33. Preserve preview/dry-run/apply safety for destructive workflows.

34. Update registry/menu/status/documentation/package metadata when relevant.

35. Return concrete validation steps, not vague assurances.

Failure prevention matrix

Failure	Prevention
Dead helper syndrome	Prove active call path before claiming UI work is complete.
Debrief/source mismatch	Verify current source and active UI before accepting claimed implementation state.
Core-to-lab dependency leaks	Add abstractions in Core; concrete lab logic stays in labs or bridges.
Repaint scanning	Move scan/index work into explicit cached services.
Generated docs overwrite curated docs	Generated content must be separate or fill missing drafts only.
Inherited-method noise	Index package-authored APIs first; hide or group inherited/lifecycle noise.
Context-losing help	Carry utility ID, section ID, topic ID, and selected context into help/documentation surfaces.
Broken persistence/domain reload	Persist and safely clamp selected topic, search, filters, splitters, selected tabs, and foldouts.
Hidden developer tools	Register explicit Developer Mode filters and visibility metadata.
Full-bundle-only implementation	Test Core-only and Core-plus-one-extension configurations.

22. AI Output, Briefing, and Debrief Standards

For audit responses
- Summarize current state from source evidence.
- Map design-bible alignment and gaps.
- Rank issues as release blocker, high-value hardening, UX polish, or future enhancement.
- Avoid claiming implementation state from debriefs alone.
- Provide a recommended implementation pass when useful.

For implementation briefs
- State primary objective.
- State relevant doctrine.
- List target files or discovery instructions.
- Define package ownership and dependency boundaries.
- Define UI/UX requirements, performance requirements, safety requirements, and optional dependency behavior.
- Include validation steps.
- Use one consistent paste-ready format throughout.

For completed implementation summaries
- Summary of what changed and why.
- Files changed.
- Architecture notes.
- UI/UX notes.
- Safety/performance notes.
- Validation steps performed.
- Known limitations.

Shared scan task-routing rule

Any feature that scans project-wide data, validates package content, indexes tokens/docs/scripts, or reports findings must
first route through shared scan contracts or explicitly justify why it remains local. If reusable by another tool, register a
provider.

23. Package Hardening Rules

Namespaces
- Use stable generic namespaces rooted in PungentFunk.Utilities or the chosen release namespace.
- Keep labs separated by namespace.
- Keep project adapters outside generic namespaces.
- Avoid names that imply a specific game project unless the code is an adapter.

Assembly definitions
- Runtime and Editor assemblies must be separated.
- Core assemblies must not reference lab assemblies.
- Bridge assemblies may reference both sides and should be removable.
- Use define constraints/version defines for optional third-party integrations.
- Validate compile with optional packages absent.

Documentation standard
- Every shipped package needs README, setup notes, known limitations, menu paths, package dependencies, and example workflows.
- Every utility descriptor should link to a help topic or documentation page.
- Every extension should document required and optional dependencies.
- Every bridge should document both sides and missing-package behavior.
- Every migration should document source data, target data, reversibility, and validation.

24. Versioning, Review, and Maintenance

Review cadence
Update this bible when doctrine changes. Update snapshot audits when source changes. Update package maps when distribution boundaries change. Update lab specs when product workflows or package ownership changes.

Changelog policy
- Record user-facing changes, package boundary changes, migration behavior, breaking changes, and known limitations.
- Separate doctrine/documentation changes from implementation changes.
- Do not use changelog entries as proof that implementation is active.

Decision log policy
Major package boundary, Core API, scan pipeline, Authoring Data Foundation, migration, or distribution decisions should be recorded in a durable decision log with rationale, alternatives considered, and validation expectations.

25. Utility Definition of Done

- Has a stable registry descriptor and menu/access surface.
- Declares package ownership, package tier, required dependencies, optional dependencies, capabilities, extension points,
  and missing dependency behavior.
- Compiles in intended configurations.
- Preserves runtime/editor separation.
- Does not add Core-to-lab dependency leaks.
- Uses shared theme/layout/state helpers where appropriate.
- Keeps scan/apply/mutation outside repaint.
- Provides preview/dry-run and Undo where needed.
- Handles empty, missing dependency, partial data, and error states.
- Is visible, reachable, usable, and verified in active UI.
- Has help/documentation entry or topic link.
- Has concrete validation steps.

26. UI Refactor Definition of Done

- Before/after control inventory completed.

- No required workflow hidden without approved replacement or documented Developer Mode gating.
- Active call path verified.
- No stale duplicate panel remains active.
- Narrow, comfortable, wide, short, tall, docked, and floating states checked.
- Splitters clamp and drag in intuitive directions.
- Search/filter/foldout/splitter state persists and restores safely.
- Empty/partial/populated states are readable.
- Help/context links preserve utility, section, topic, and selected item where relevant.
- No scan/index/reflection work runs during routine repaint.
- Missing optional package states are calm and non-breaking.

27. Authoring and Package Split Review Gates

Authoring Foundation gate

- Does the feature use shared authoring IDs instead of inventing new permanent IDs?
- Does it use shared link targets where possible?
- Does it expose shared metadata, preview, actions, and validation results?
- Does it behave safely when the concrete editor package is missing?
- Does it preserve legacy note data and relationships?
- Does it avoid Core dependencies on concrete editor windows?

Package split gate

- Which package owns this feature?
- Is the feature Core contract, extension workflow, bridge glue, full bundle convenience, or project adapter?
- What dependencies are required?
- What dependencies are optional?
- What capabilities are provided and consumed?
- What extension points are provided and consumed?
- What UI appears when optional packages are missing?
- Can the package compile with only Core plus its declared required dependencies?
- Could this accidentally make the full bundle the only working configuration?

Notes and Roadmap migration gate

- Does the pass preserve existing note IDs, metadata, tags, targets, links, and integrations?
- Does it preserve context menus, inspector note summaries, scene badges, help bridges, audit bridges, token links, and documentation-link relationships?
- Does it deprecate only the old freeform writing panel and not the Notes system itself?
- Does it provide read-only selected item preview in the browser?
- Does it route editing to Rich Document Editor when available?
- Does it show safe missing-editor states when unavailable?

Parallel development readiness gate

Concurrent Rich Document, Board/Graph, and Data Sheet development may begin only when Authoring Data Foundation contracts compile, a Legacy Note provider exists, the Authoring Browser can display provider records, and missing-provider behavior is visible and non-breaking.

Recommended implementation pass sequence

36. Authoring Data Foundation contracts.

37. Legacy Notes provider adapter.

38. Authoring Browser shell conversion.

39. Registry package metadata expansion.

40. Rich Document Editor MVP.

41. Board/Graph MVP.

42. Data Sheet MVP.

43. Optional bridges in priority order.

44. Package split validation matrix.

Minimal package configuration matrix

Configuration	Expected result
Core only	Registry, theme, utilities browser, minimizer, help/docs read/open, scan contracts, authoring contracts, lightweight Authoring Browser shell compile and open. Missing concrete editors show disabled actions.
Core + Documentation and Authoring	Rich documents can be edited; legacy notes can be previewed and migrated/linked.
Core + Board/Graph	Board documents can be browsed and edited; document/sheet links degrade gracefully if missing.
Core + Data Sheet	Sheets can be edited/imported/exported; audit/token bridges degrade if missing.
Core + Audit/Validation	Advanced audit UI and providers work; authoring/sheet export actions degrade if missing.
All extensions without bridges	Each package works independently with Core; bridge-only actions are absent or disabled.
Full Bundle with bridges and	All official cross-package workflows are enabled

discoverable.

28. Reference Basis

This edition is based on the Shared Systems and Project Scan Pipeline Doctrine edition, the current package-split and Authoring Foundation planning brief, and the ongoing PungentFunk Utilities roadmap through Token Validator, Notes/Authoring, Image/Media Token System, Developer Mode tiers, rich authoring, whiteboard/board documents, project/session reports, and spreadsheet/data-sheet utilities.

Explicit answers encoded into doctrine

Question	Doctrine answer
Should Notes Browser remain in Core/free or move to Core/free. Move Documentation and Authoring?	Keep a reduced Authoring Browser shell in Core/free. Documentation and advanced authoring/editing features to Authoring.
Should Token Validator core parsing be in Core/free while advanced scan UI lives elsewhere?	Yes. Token contracts/parser primitives belong in Core when shared. Advanced project scan UI belongs in Audit/Validation, with authoring bridges.
Should Documentation Links editing remain developer-authoring only?	Safe read/open belongs in Core. Editing belongs in Documentation and Authoring, with risky/generated authoring controls gated as needed.
Should Coverage Matrix be deprecated into Data Sheet?	Yes. Keep compatibility aliases, then evolve it into a Data Sheet view.
What is the minimal package set needed to browse notes/documents/boards/sheets?	Core/free, through the Authoring Browser shell and provider contracts. Concrete editing requires the relevant extension.
How should missing editor packages appear?	Browseable records with metadata, fallback preview, disabled edit action, package/install hint, and safe copy/open raw actions.
How should extension packages register capabilities without hard registries, service dependencies?	Through Core descriptor metadata, provider lookup, asmdef constraints, version defines, and reflection-safe bridges.
How should PungentNote.body migrate?	Preserve as legacy read-only content, create linked RichDocument copies, record migration state, keep backlinks, and avoid destructive conversion.
How do we prevent full-bundle-only behavior?	Validate Core-only and Core-plus-one-extension configurations, bridge-disabled states, and missing-package UI.

29. Closing Doctrine

PungentFunk Utilities should feel cohesive because its packages share contracts, visual language, registry semantics, help/documentation behavior, scan infrastructure, authoring identity, and safety rules. It should not feel cohesive because every feature has been forced into Core or because the full bundle is the only tested configuration.

Core should remain small, stable, and boring. Extensions should be valuable on their own. Bridges should make combinations better without making standalone packages fragile. The Authoring Data Foundation should make notes, documents, boards, sheets, tokens, help topics, audit issues, utilities, and external targets interoperable without erasing the distinction between browsing, editing, graphing, and tabular authoring.

A pass is successful only when the current source structure and active Unity UI verify the claim. A package split is successful only when missing optional packages are calm, visible, and non-breaking. An authoring system is successful only when legacy notes remain safe while richer document, board, and sheet editors can develop independently on the same foundation.

Documentation, Learning Surface, and Information Architecture Doctrine

Help and documentation surfaces are first-class product UI. They preserve breadth while making it easier to discover, read, navigate, and maintain the PungentFunk Utilities suite.

Product learning surface

- Do not reduce feature breadth just to reduce visual clutter.
- Use structured information architecture to reveal breadth gradually.
- Help Browser follows documentation-site patterns: landing page, search, left navigation, readable article body, optional right context rail, breadcrumbs, related links, and contextual overlay trays.

Documentation layout

- Topic pages should read like articles: overview, when to use, quick start, controls, safety notes, examples or scripting/API, troubleshooting, and related material.
- Use the right rail for on-page contents, related topics/utilities, notes, documentation links, source badges, and generated/developer metadata.

Dashboard and generation surfaces

- Separate status, action, review, and raw diagnostics.
- Count chips must be tooltip-rich, actionable, or visually subordinate.
- Developer maintenance UI must be collapsible, scrollable, and resizable where panels compete for space.
- Summary cards show the few metrics that guide the next action; raw counts belong in details, queues, or exports.

Package-wide extrapolation

Any utility with dense status or generation data must use summary first, task-path cards, collapsible details, persistent filters, actionable empty states, tooltip-rich metadata, and clear separation between public controls and developer-only maintenance controls.

Quick help and popup-style documentation should prefer overlay trays when the task is contextual and lightweight.